CAPE: Critical-Path-Aware Prefetching for Fast and Energy-Aware Edge–Cloud Speculative Decoding

Zishi Deng
University of Auckland, Auckland, New Zealand
zden973@aucklanduni.ac.nz

***Abstract*—Speculative decoding splits naturally between the edge and the cloud: a small draft model on the device proposes tokens and a large cloud model verifies them. Every verification, however, costs a network round trip. Drafting the next block while the cloud verifies the current one (lookahead) hides this latency, but it spends scarce device energy on tokens that are discarded whenever the verifier rejects the speculated prefix. Measurements on a Jetson Orin Nano that drafts over campus Wi-Fi for a Llama-3.1-8B verifier on an RTX 3080 server show that drafting occupies 37%–47% of a synchronous round, that verification time barely grows with draft length (0.45 ms per draft token against a mean of 96 ms), and that board energy is predicted by wall time and draft-busy time (R^2 = 0.990). We present CAPE, an edge-only controller that keeps fresh drafting off the critical path. CAPE models decoding as a two-state Markov chain over prefetch hits and misses. In every round it selects the fresh draft window and the lookahead length that minimize an energy-weighted time per expected token, using online estimates of acceptance, round-trip time and draft cost. CAPE keeps strict greedy verification and requires no change to the cloud server. On 110 prompts from GSM8K, HumanEval and MT-Bench (1,540 measured requests), CAPE's speed setting raises throughput by 9.9% [8.0, 12.0] over the fastest fixed-window configuration, at 3.2% more energy per token. It is also faster than the best fixed window chosen separately for each network delay, by 6.2% and 6.6%. Its energy setting matches the fixed window's speed with 6.1% [4.5, 7.7] lower energy per token. CAPE is 31.4%–43.4% faster than synchronous speculative decoding and 32.9%–45.1% faster than AdaEDL.**

***Index Terms*—speculative decoding, edge–cloud collaboration, large language models, energy efficiency, embedded GPU**

I. Introduction

Large language models (LLMs) increasingly run close to their users, on phones, robots and embedded boards, for responsiveness, privacy and cost. Such devices can host only small models, while the quality users expect comes from models with billions of parameters running in the cloud. Speculative decoding [1], [2] offers a natural split between the two. A small draft model proposes several tokens, and the large target model checks all of them in one forward pass. With greedy verification every emitted token is the target model's own greedy choice, yet each target pass can emit several tokens. Placing the draft model on the device and the target model in the cloud combines local responsiveness with cloud-level quality, and several recent systems explore this split [3]–[7].

The split exposes a cost that single-machine speculative decoding does not have: every verification is a network round trip. In a synchronous edge–cloud loop, the device drafts, sends the draft and waits idle for the verdict before drafting again. A natural remedy is *lookahead*. While the cloud verifies the current block, the device already drafts the next one, assuming that the current block will be accepted; parallel drafting and verification has been explored on a single server [8]. Lookahead turns idle waiting into useful work, but it creates two tensions that matter at the edge. First, the speculated continuation is usable only if the verifier accepts the entire block *and* its bonus token matches the draft's guess. Otherwise it is discarded, and on an energy-constrained device every discarded token is wasted energy. Second, the right amount of lookahead depends on quantities that change across requests and rounds: the network delay, the verification time, the draft speed and how predictable the text currently is.

We study these tensions on a real deployment: an NVIDIA Jetson Orin Nano drafting with Llama-3.2-1B over campus Wi-Fi for a Llama-3.1-8B verifier served by vLLM [9] on an RTX 3080 server, with the board's input power measured for every request. Three observations shape our design (Section III):

- **Drafting sits on the critical path.** In synchronous speculative decoding, local drafting takes 47% of a round with 2 ms of added delay and 37% with 100 ms.
- **Verification cost is almost flat in the draft length.** Each additional draft token adds only 0.45 ms to a mean verification time of 96 ms, because the cost is dominated by recomputing the context.
- **Draft work has a predictable energy price.** Per-request board energy is explained by wall time and draft-server busy time (R^2 = 0.990), so wasted drafting translates directly into joules.

Together, these observations suggest a simple principle. *Tokens drafted after a verification returns delay the next request, whereas tokens drafted during verification are nearly free in time but not in energy.* A good controller should therefore draft as little as possible on the critical path, and draft ahead only as much as is likely to be reused.

We present CAPE (Critical-path-Aware PrEfetching), an edge-only controller built on this principle. CAPE models the decoding pipeline as a two-state Markov chain whose states record whether the current round can reuse a prefetched block (a hit) or must draft afresh (a miss). For every pair of fresh draft window γ_f and lookahead length h, the model predicts the long-run time, draft-busy time and number of

emitted tokens per round. CAPE picks the pair that minimizes an energy-weighted cost per token, where a single weight w trades speed against energy. Its inputs are estimated online from the last few rounds: state-dependent acceptance rates, the verification round trip and the per-token draft cost. CAPE keeps strict greedy verification, so every emitted token is the target model's top choice, and it requires no change to the cloud server. The controller adds 170 lines of Python to a lookahead client and decides in about 0.42 ms per round.

We evaluate CAPE on all 110 prompts of our GSM8K, HumanEval and MT-Bench sets at two network delays, against synchronous decoding, three fixed windows and AdaEDL [10]: 1,540 measured requests with randomized interleaving and paired prompt-cluster bootstrap confidence intervals. With its speed setting, CAPE is 9.9% [8.0, 12.0] faster than the fastest fixed window, and faster on every task. It even beats an oracle that picks the best fixed window for each delay, by 6.2% at 2 ms and 6.6% at 100 ms, while using less energy per token than that oracle at both delays. With its energy setting, CAPE keeps the speed of the fastest fixed window and lowers J/token by 6.1% [4.5, 7.7]. It is 31.4%–43.4% faster than synchronous speculative decoding and 32.9%–45.1% faster than AdaEDL, and its energy setting is both faster and more energy-efficient than AdaEDL and an eight-token fixed window. On two earlier held-out prompt sets, the default setting was 4.8% and 6.0% faster than the fixed window without increasing energy per token.

In summary, this paper makes the following contributions:

- A measurement study of edge–cloud speculative decoding on an embedded GPU. It quantifies the critical-path share of drafting and the near-constant cost of verification, and it fits a per-request board energy model (Section III).
- CAPE, a critical-path-aware lookahead controller based on a two-state Markov model with an explicit energy price. It runs entirely on the device and keeps strict greedy verification (Section IV).
- An evaluation on real hardware with board-level energy measurement: 1,540 requests on 110 prompts against synchronous decoding, three fixed windows and AdaEDL, plus two held-out prompt sets and a heuristic latency-regime controller (Section VI).

II. Related Work

Speculative decoding. Leviathan et al. [1] and Chen et al. [2] introduced draft-then-verify decoding with verification rules that preserve the target model's output distribution; Xia et al. [11] survey the many variants that followed. One line of work raises the number of tokens accepted per target pass. SpecInfer organizes the candidates of small speculative models into a token tree that the target verifies in parallel [12]. Sequoia constructs token trees by dynamic programming and adapts their size and depth to the hardware [13]. BiLD lets a small model generate most tokens and invokes the large model only when needed [14]. Medusa [15] and EAGLE [16] remove the separate draft model by adding decoding heads to the target or drafting at the feature level. Lookahead decoding [17] parallelizes decoding without any draft model; despite its name, it is unrelated to the draft-ahead lookahead studied here. These techniques assume that both models share one machine. We use strict greedy verification with an independent draft model on the device. CAPE changes only the drafting schedule around the network round trip, which is complementary to techniques that raise acceptance.

Adaptive draft length. AdaEDL [10] stops drafting when an entropy-based lower bound on the acceptance probability becomes small, and SpecDec++ [18] trains an acceptance-prediction head to decide when to stop. These methods decide how many tokens to propose in a co-located setting where drafting and verification alternate. CAPE instead decides how many tokens to draft on and off the critical path of an edge–cloud pipeline. It needs neither draft probabilities nor training, and it prices the device energy of speculative work explicitly. Our comparison with AdaEDL (Section VI-B) shows why the distinction matters: when verification cost is nearly independent of the draft length, stopping early saves little.

Parallel drafting and verification. PEARL [8] combines pre-verification and post-verification so that drafting and verification wait less for each other, and it adapts the draft length. Overlapping the two models is therefore not new. CAPE applies the overlap across a network and contributes a model that prices both the critical path and the energy of speculative work, for a device that pays for every discarded token.

On-device and edge–cloud speculative decoding. EdgeLLM accelerates LLM inference on mobile devices with speculative decoding [19], keeping both models on the device. Several systems instead split the models across machines. In SLED, lightweight edge devices draft with their own draft models and a shared edge server verifies their requests in batches [3]. SpecEdge drafts on consumer-grade edge GPUs, overlaps edge drafting with server verification, and interleaves the requests of many users to raise server throughput [4]. DSD studies distributed speculative decoding with a discrete-event simulator and an adaptive window policy [5]. PicoSpec runs edge and cloud models in an asynchronous pipeline and compresses the transmitted vocabulary distributions [6]. WISP suppresses wasted drafting and verification interference with dynamic drafting and SLO-aware batching [7]. Lyu et al. [20] show analytically that pipelining makes distributed speculative decoding competitive mainly when the round trip is short relative to edge drafting, and that its main advantage is server capacity rather than single-request latency. These systems mostly target server-side throughput, multi-user scheduling or communication volume. CAPE targets the critical path and the energy of a single device: it runs entirely on the device, treats the verifier as a black box, and decides per round how much drafting to expose and how much to hide, which is exactly the trade-off that determines whether pipelining pays off.

Energy. Samsi et al. [21] benchmark the energy cost of LLM inference on data-center GPUs. Dutta et al. [22] show that the speedup of speculative decoding does not translate proportionally into energy savings, which motivates measuring energy directly rather than treating speed as its proxy. We

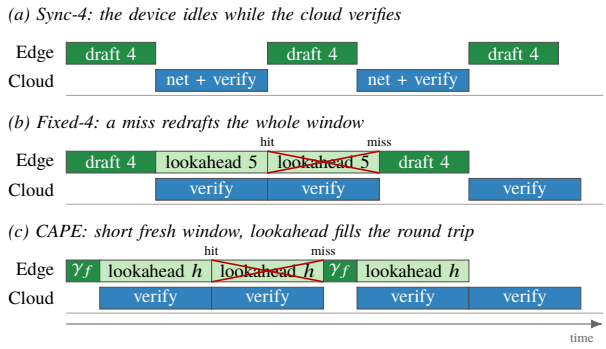


Fig. 1. Round timelines (schematic). Fresh drafting (dark) delays the next verification; lookahead (light) overlaps it and is reused after a hit or discarded after a miss (crossed). CAPE drafts few fresh tokens and sizes the lookahead to the round trip.

measure the board energy of the edge device and use a fitted energy model inside the controller.

III. Background and Measurement Study

A. Edge–cloud speculative decoding

Let x be the current prefix. In each round the device drafts γ tokens $D = (d_1, \ldots, d_\gamma)$ with the draft model and sends them to the cloud. The target model scores $x \| D$ in one forward pass and returns the number n of leading draft tokens that equal its own greedy predictions, together with one target token t. This token is the correction at the first mismatch, or a bonus token if all γ draft tokens are accepted. The prefix becomes $x \| d_{1:n} \| t$, so each round emits $n + 1$ tokens. We use strict greedy acceptance: a draft token is accepted only if it equals the target's highest-scoring token, so every emitted token is the target model's greedy choice.

Lookahead. While the cloud verifies D, the device can draft h further tokens $L = (\ell_1, \ldots, \ell_h)$ from $x \| D$, assuming full acceptance; ℓ_1 is the draft model's guess of the bonus token. If $n = \gamma$ and $t = \ell_1$, then $\ell_{2:h}$ is a valid draft for the next round and can be sent at once (a *hit*). Otherwise L is discarded (a *miss*). A fixed window with $h = \gamma + 1$ prefetches exactly one complete next block (Fig. 1(b)).

B. Testbed

Edge device. The edge device is an NVIDIA Jetson Orin Nano Super developer kit (8 GB) in its 25 W power mode, running JetPack 6.2.1 (L4T R36.4.7). It serves Llama-3.2-1B-Instruct [23], quantized to 4 bits (Q4_K_M), with llama.cpp (build b9744) as a single-slot HTTP server with a 4,096-token context and all layers on the GPU. Board input power is sampled at 10 Hz from the INA3221 VDD_IN rail and integrated over each request. It includes idle and operating-system power, but not server or network energy.

Server. A workstation stands in for the cloud: an AMD Ryzen Threadripper PRO 5965WX CPU (24 cores), 125 GiB of memory and an NVIDIA GeForce RTX 3080 GPU with 10 GB of memory, running Ubuntu 24.04 with NVIDIA driver 580.173.02. It serves Llama-3.1-8B-Instruct, quantized to 4 bits with AWQ [24], with vLLM 0.26.0 [9] and PyTorch 2.11.0, with a 4,096-token context. The server ran no other workload during our experiments. A small token-ID verification API in front of vLLM requests top-1 prompt log-probabilities; it is identical for all methods.

Network. Both the Jetson and the server are connected to the campus Wi-Fi network, so every verification crosses two wireless links. This real path is fast but irregular: across the 107,200 verification rounds of our main evaluation, transport and HTTP overhead per request had a median of 18 ms, a 90th percentile of 115 ms and a 99th percentile of 178 ms, and exceeded 100 ms in 12% of rounds. To study slower paths without privileged access, the client adds a fixed delay of 2 ms or 100 ms to every verification request, on top of this real path. The measurements in this section are taken from the 1,540 requests of our main evaluation (Section VI), which covers seven decoding methods.

C. Observation 1: drafting is on the critical path

Fig. 2(a) decomposes the mean round of synchronous speculative decoding with four draft tokens (Sync-4). With 2 ms of added delay, local drafting takes 124 ms of a 265 ms round (47%). With 100 ms, it takes 142 ms of 379 ms (37%). Cloud verification takes about 98 ms in both cases. A round yields only 3.7 tokens on average, so drafting time directly limits throughput, and any drafting moved into the shadow of verification shortens the round.

D. Observation 2: verification cost is almost flat in the draft length

Fig. 2(b) shows the verification time reported by the cloud server against the number of draft tokens sent. A regression with per-request intercepts over 107,200 rounds attributes 0.31 ms to each context token but only 0.45 ms to each draft token, against a mean of 96 ms; the mean rises only from 88 ms with one draft token to 100 ms with eight. The verifier must return target log-probabilities at every draft position, and in our deployment vLLM does not reuse cached prefixes for requests that ask for prompt log-probabilities. Every verification therefore recomputes the whole context, and a few more draft positions hardly matter. Sending more draft tokens per round is almost free for the cloud; the cost of a draft token is paid on the device.

E. Observation 3: draft work has a predictable energy price

Background draft calls take 24.9 ms plus 20.2 ms per token ($R^2 = 0.95$ over 46,388 calls). Per-request board energy fits

$$E \approx 3.25\,\mathrm{W} \cdot T + 10.56\,\mathrm{W} \cdot B, \tag{1}$$

where T is the request's wall time and B the draft server's busy time ($R^2 = 0.990$, mean absolute percentage error 5.1%; Fig. 2(c)). A draft token thus costs about 0.21 J above base power, a sizable share of the 0.65–0.98 J spent per output token. Speculative work that is discarded is visible in the energy bill. An eight-token fixed window discards 120 draft tokens per 100 output tokens and uses 30.9% more energy per token than a four-token window (Section VI).

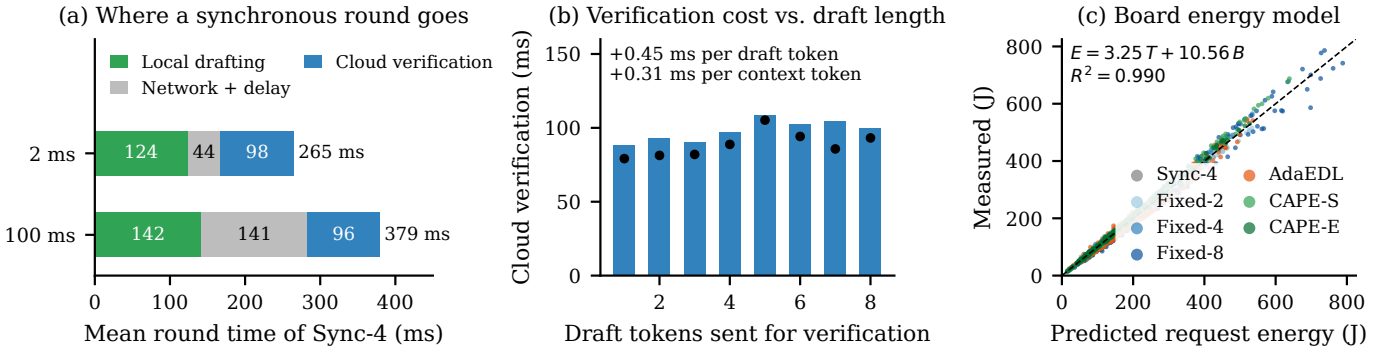


Fig. 2. Measurement study. (a) Mean round of synchronous speculative decoding with four draft tokens (Sync-4): drafting is 37%–47% of the critical path. (b) Cloud verification time by the number of draft tokens sent (bars: mean, dots: median; 107,200 rounds); with per-request intercepts, each draft token adds 0.45 ms and each context token 0.31 ms. (c) Measured request energy against the fit of Eq. (1) (1,540 requests, seven methods).

F. Design implications

The observations lead to four goals. (i) Keep fresh drafting off the critical path. (ii) Fill the verification time with lookahead, but only as much as is likely to be reused, and less when energy matters. (iii) Adapt to the round trip and the acceptance rate, which vary across and within requests. (iv) Leave the cloud server and the verification rule unchanged.

IV. CAPE Design

A. Overview

At the start of each round, CAPE chooses a fresh window γ_f and a lookahead length h (Algorithm 1). If the previous lookahead is reusable, CAPE sends all $h-1$ reusable tokens at once and drafts fresh tokens only if fewer than γ_f remain, so no drafting delays the request. Otherwise it drafts γ_f fresh tokens on the critical path, usually only one or two. As soon as the request is sent, CAPE drafts h lookahead tokens in the background while the cloud verifies. Short fresh windows keep rounds short after misses, and long lookahead converts verification time into ready-to-send drafts after hits (Fig. 1(c)).

B. Two-state pipeline model

Let c and d be the per-call and per-token draft costs, V the verification round trip from sending a draft to receiving the verdict, and α_M and α_H the per-token acceptance probabilities in miss and hit rounds. For a candidate (γ_f, h), background drafting takes $b = c + dh$. A round ends only when both verification and lookahead have finished, which takes $o = \max(V, b)$ after the request is sent.

Miss rounds draft γ_f tokens first:

$$T_M = c + d\gamma_f + o, \qquad B_M = c + d\gamma_f + b,$$
$$N_M = \textstyle\sum_{i=0}^{\gamma_f} \alpha_M^i, \qquad p_{MH} = \alpha_M^{\gamma_f+1}. \tag{2}$$

Here N_M is the expected number of emitted tokens (accepted tokens plus the target token) under geometric acceptance, and p_{MH} is the probability that all γ_f tokens and the bonus guess are accepted, which makes the next round a hit.

Hit rounds send $s = \max(h-1, \gamma_f)$ tokens, of which $u = s - (h-1)$ must be drafted as a top-up:

$$T_H = [u > 0](c + du) + o, \qquad B_H = [u > 0](c + du) + b,$$
$$N_H = \textstyle\sum_{i=0}^{s} \alpha_H^i, \qquad p_{HH} = \alpha_H^{s+1}. \tag{3}$$

The stationary hit share of the chain is $\pi_H = p_{MH}/(p_{MH}+1-p_{HH})$. We write $\bar{X} = (1-\pi_H)X_M + \pi_H X_H$ for $X \in \{T, B, N\}$.

Objective. By Eq. (1), board energy is proportional to $T + rB$, where r is the ratio of draft-active to base power. CAPE minimizes

$$J(\gamma_f, h) = \frac{\bar{T} + w\,(\bar{T} + r\bar{B})}{\bar{N}}, \tag{4}$$

which is the time per token plus w times the energy per token, expressed in seconds of base power. With $w = 0$, CAPE optimizes speed only. Larger w makes speculative work more expensive and shortens the lookahead when acceptance is low.

The model captures the trade-offs of Section III. A larger γ_f yields more tokens per miss round but lengthens the critical path. A larger h costs no time while $b \le V$ and makes a hit more valuable, but it raises B in every round, hit or miss. Separate acceptance rates for the two states capture the correlation of acceptance across rounds: hits happen in predictable stretches of text, where the next block is also more likely to be accepted. In our main evaluation, late-request estimates of α_H average 0.84, against 0.79 for α_M.

C. Online estimation

CAPE re-estimates its inputs before every round and resets them for every request. $\hat{\alpha}_H$ is estimated from rounds that follow a fully accepted round, the precondition of a hit, and $\hat{\alpha}_M$ from all other rounds. Each estimate uses the last 32 such rounds and a prior of 8 accepted tokens out of 10 examined:

$$\hat{\alpha}_s = \min\left(0.99,\ \frac{8 + \sum_j n_j}{10 + \sum_j \left(n_j + [n_j < \gamma_j]\right)}\right), \tag{5}$$

where round j of state s sent γ_j tokens and n_j of them were accepted. $\hat{V}$ is the median of the last three verification round trips (150 ms before the first one completes). It follows a change in delay within a few rounds while ignoring single spikes. The per-token draft cost $\hat{d}$ is the median, over recent

Algorithm 1 One CAPE round on the device

Require: prefix x; pending lookahead L drafted from x'; estimates $\hat{\alpha}_M, \hat{\alpha}_H, \hat{V}, \hat{d}$

$(\gamma_f, h) \leftarrow \arg\min J(\gamma_f, h)$ ▷ Eq. (4)
if $x = x' \| \ell_1$ **then** ▷ hit
  $D \leftarrow \ell_{2:h}$, topped up to γ_f tokens
else ▷ miss
  draft γ_f tokens D from x ▷ critical path
end if
send (x, D) to the verifier
draft h tokens L from $x \| D$ in the background
receive (n, t); $x \leftarrow x \| d_{1:n} \| t$
update $\hat{\alpha}_M$ or $\hat{\alpha}_H$, $\hat{V}$ and $\hat{d}$

background calls, of the call duration minus the overhead $c = 10$ ms, divided by the number of tokens drafted (24.5 ms before the first call). The power ratio is fixed at $r = 2.3$, calibrated on development runs.

D. Decision and execution

CAPE evaluates Eq. (4) for all $\gamma_f \in \{1, \ldots, \Gamma\}$ and $h \in \{2, \ldots, \Gamma + 1\}$ with $\Gamma = 8$. These 64 candidates take about 0.42 ms in Python on the Jetson CPU, less than 0.2% of a round. Lookahead is capped by the remaining token budget and skipped in the last round. Lookahead still pending at the end of a request is awaited and charged to its wall time and energy.

Exactness. CAPE changes only which draft tokens are proposed, never how they are verified. A prefetched block is reused only if the full verified prefix, including the bonus token, equals the prefix from which it was drafted. Every emitted token is therefore the target model's top choice for its prefix. In exact arithmetic, CAPE would produce the same output as any other strict greedy configuration. With a quantized target served in floating point, near-tied scores can resolve differently when the verification computation has a different shape; Section VI-E quantifies this effect.

V. Implementation

CAPE is implemented in the Python edge client of our testbed (Section III-B). A lookahead client of 163 lines manages background draft calls to llama.cpp on a worker thread. It checks prefix compatibility and records every speculative token as reused, discarded or unused. CAPE adds the pipeline model (49 lines) and the controller (121 lines) on top of it. Draft and target exchange token IDs, so no text is re-tokenized between rounds, and the draft server is not asked for token probabilities. The cloud verification service is a FastAPI front end to vLLM that returns the target's top-1 token at every draft position; it is identical for all methods. Every request logs per-round timings, all speculative events, every controller decision and the power trace, which allows each number in this paper to be recomputed from raw records.

VI. Evaluation

We ask four questions. How much faster is CAPE than fixed windows, synchronous decoding and AdaEDL, and at what energy cost (Section VI-B)? Where do its gains come from (Section VI-C)? Do the results hold across tasks, prompts and evaluation sets (Section VI-D)? And how do exactness, task accuracy and overhead behave (Section VI-E)?

A. Methodology

Workloads. Prompts come from our local subsets of GSM8K [25] (50 grade-school math problems), HumanEval [26] (30 Python function completions) and the first-turn questions of MT-Bench [27] (30 open-ended writing and reasoning prompts). Generation uses the Llama 3 chat template, greedy decoding and at most 512 output tokens. Every request runs once with 2 ms and once with 100 ms of added delay per verification.

Evaluation sets. The main evaluation runs all seven methods below on all 110 prompts at both delays, 1,540 measured requests in total. Before it, CAPE's default setting was evaluated on two held-out sets of 9 and 12 prompts (72 requests each), against Fixed-4, a heuristic latency-regime controller and a variant with more elaborate estimators. Neither the held-out sets nor the main evaluation were used to design or tune CAPE. The held-out sets ran with the draft server configured with four slots and an 8,192-token context; single-stream draft speed was unchanged.

Protocol. For each evaluation, the methods, prompts, seeds, delays and analysis were frozen in a plan file before the first measured request. Methods are interleaved in a randomized, balanced order within each prompt–delay block. Reference outputs and equal 16-token warm-ups are generated before timing starts, and no request is retried. The main evaluation was interrupted once by a dropped connection to the cloud server and resumed with the same plan; the interrupted request was run again. Throughput is the total number of output tokens divided by the total wall time, including terminal lookahead. Energy per token is the integrated board energy divided by the number of output tokens. Paired effects come with 95% percentile intervals from 2,000 bootstrap resamples of prompts. A positive J/token saving means less energy per token.

Methods. All methods share the models, servers, token-ID transport and strict greedy verification.

- **Sync-4**: synchronous speculative decoding with four draft tokens per round and no lookahead.
- **Fixed-γ**, $\gamma \in \{2, 4, 8\}$: γ draft tokens per round with a complete-block lookahead of $h = \gamma + 1$. Fixed-4 is the fastest fixed window pooled over both delays and the reference for all relative numbers.
- **AdaEDL** [10]: synchronous decoding that streams up to eight draft tokens and stops before the first token whose entropy-based acceptance bound $1 - \sqrt{0.2\,H}$ falls below an adaptive threshold. We use the paper's update rule and hyper-parameters. The entropy is computed from the top-20 draft probabilities plus the residual mass. The initial

threshold, which the paper does not specify, is set to 0.5, the best of five values on development prompts.
- **CAPE** with $w = 0$ (CAPE-S, speed), $w = 0.25$ (CAPE, default; held-out sets) and $w = 0.5$ (CAPE-E, energy).
- **Regime** (held-out Set 1 only): a latency-regime heuristic from our earlier work. After a four-token warm-up, it chooses among complete-block windows $\gamma \in \{2, 4, 6, 8\}$ from the measured verification interval and admits longer windows only when the measured overlap allows them.

B. Main results

Table I and Fig. 3 summarize the main evaluation; Fig. 4 shows the paired effects with their intervals.

Lookahead is essential across a network, but no fixed window fits all delays. Fixed-4 is 30.5% [28.0, 33.2] faster than Sync-4, at the price of 15.0% more J/token. Fixed-4 is the fastest fixed window at 2 ms. At 100 ms, Fixed-8 is 5.5% faster than Fixed-4, but it uses 23.8% more energy per token there and is 19.6% slower at 2 ms. Fixed-2 emits too few tokens per round and is 18.8% slower than Fixed-4 overall.

Adapting the draft length alone does not help here. AdaEDL stops drafting when acceptance becomes unlikely, which saves target computation when verification cost grows with the draft. In the edge–cloud setting, verification cost is nearly independent of the draft length (Observation 2), and the device idles during every round trip. AdaEDL is therefore not faster than Sync-4 (−1.2% [−3.2, 0.9]) and uses 12.4% more J/token, because it drafts longer blocks and streams draft probabilities.

CAPE-S is the fastest method. CAPE-S raises throughput by 9.9% [8.0, 12.0] over Fixed-4. At 2 ms it is 6.2% [3.6, 8.9] faster *and* uses 4.9% [2.9, 7.0] less energy per token. At 100 ms it is 12.5% [10.2, 14.9] faster, buying speed with 10.4% more J/token; over both delays its J/token is 3.2% higher. CAPE-S also beats an oracle that picks the best fixed window for each delay. It is 6.2% faster than Fixed-4 at 2 ms and 6.6% [4.6, 8.6] faster than Fixed-8 at 100 ms, where it also uses 10.9% less energy per token. It is 43.4% faster than Sync-4 and 45.1% faster than AdaEDL.

CAPE-E saves energy at the speed of the fastest fixed window. CAPE-E matches Fixed-4's throughput (+0.7% [−1.3, 3.1]) and lowers J/token by 6.1% [4.5, 7.7], by 10.5% at 2 ms. It dominates AdaEDL (+32.9% throughput, +3.9% J/token saving) and Fixed-8 (+7.7% and +28.2%), with all four intervals excluding zero. It is 24.0% faster than Fixed-2 at equal energy (+0.7% [−0.6, 2.0]). Compared with Sync-4, the most frugal method, CAPE-E is 31.4% faster for 8.0% more J/token.

One weight selects the operating point. Raising w from 0 to 0.5 trades 8.4% of throughput for 9.0% lower J/token. Both settings lie on the speed–energy frontier of Fig. 3: no other method is both faster and more frugal than either of them at either delay.

C. Where the gains come from

Fig. 5 shows the decisions behind these results. CAPE keeps the critical path short: the mean fresh window of CAPE-S is 1.6 tokens at 2 ms and 2.4 at 100 ms, against four for Fixed-4. It sizes the lookahead to the round trip: the mean lookahead of CAPE-S grows from 4.9 to 8.4 tokens when the delay increases. CAPE-E prices speculative work and grows its lookahead only from 3.8 to 4.9 tokens. This behavior emerges from Eq. (4) and the online estimates; no delay threshold is configured.

The lookahead is also better used. CAPE-S and CAPE-E reuse 53% and 56% of their lookahead blocks, against 49% for Fixed-4. CAPE-E discards 55 draft tokens per 100 output tokens, against 67 for Fixed-4, and keeps the draft server busy for 44.4 ms per output token instead of 48.3 ms, which explains its energy saving through Eq. (1). CAPE-S spends more draft time (52.1 ms per token) to buy speed, as instructed by $w = 0$.

D. Robustness across tasks, prompts and sets

Tasks. Table II breaks the effects down by task. CAPE-S is faster than Fixed-4 on every task: by 8.9% on GSM8K, 17.8% on HumanEval and 5.7% on MT-Bench, with all intervals above zero. MT-Bench has the lowest acceptance rate (0.76 with Fixed-4, against 0.88 for GSM8K and 0.92 for HumanEval). There, CAPE-S pays for its speed with 6.5% more J/token, whereas CAPE-E saves 11.7% at 5.0% lower throughput because it stops prefetching blocks that are unlikely to be used. On HumanEval, CAPE-E is also 11.5% faster than Fixed-4 at unchanged energy.

Prompts and host activity. Some of the 110 prompts had appeared in earlier recorded phases (the held-out sets, a pilot comparison and the AdaEDL threshold sweep). On the 80 prompts outside these phases, CAPE-S gains 9.8% [7.6, 12.3] in throughput and CAPE-E saves 5.8% [3.9, 7.8] of J/token. While the main evaluation ran, we briefly used the host for analysis and logged each period. Excluding the 6 prompt–delay blocks that overlap these periods leaves the results unchanged (9.8% faster and 6.0% less J/token).

Held-out sets. Table III shows the two held-out sets, which were run before the main evaluation. The default setting ($w = 0.25$) increased throughput over Fixed-4 by 4.8% [1.2, 9.3] and 6.0% [1.1, 10.7] without a measurable energy penalty (J/token savings of +3.7% and +3.6%). It was 3.8% [1.4, 6.2] faster than the Regime heuristic, whose own gain over Fixed-4 was +0.9% [−2.4, 4.1]. A variant with more elaborate estimators (hierarchical shrinkage of acceptance, an expected-maximum round-trip estimate and a restricted action set) changed throughput by −0.6% [−4.1, 3.8] relative to CAPE, so the simple estimators of Section IV suffice.

E. Exactness, task accuracy and overhead

All 1,540 requests completed with valid power traces. Every emitted token is the target's top choice under the verification pass, yet outputs are not always identical across methods (Table IV). In 45 of the 220 prompt–delay blocks, at least one method's output differs from the others, typically after many identical tokens (median first difference at token 170 between Fixed-4 and CAPE-S). The differences track the size of the verified window rather than the controller. Sync-4, which verifies the same four-token windows as Fixed-4, differs

TABLE I

MAIN EVALUATION: 110 PROMPTS, TWO DELAYS, 1,540 REQUESTS. THROUGHPUT AND J/TOKEN PER METHOD; PAIRED EFFECTS RELATIVE TO FIXED-4 WITH 95% PROMPT-CLUSTER BOOTSTRAP INTERVALS; DRAFT-SERVER BUSY TIME PER OUTPUT TOKEN AND DISCARDED DRAFT TOKENS PER 100 OUTPUT TOKENS.

	Throughput (tok/s)			J/token	Relative to Fixed-4, 2 and 100 ms pooled		Draft busy	Discarded
Method	2 ms	100 ms	pooled	pooled	Throughput gain (%)	J/token saving (%)	ms/token	per 100 tok.
Sync-4	13.86	9.74	11.43	0.650	−23.4 [−24.9, −21.9]	+13.1 [11.1, 15.0]	35.9	0
Fixed-2	15.94	9.78	12.11	0.708	−18.8 [−20.7, −16.8]	+5.4 [3.4, 7.4]	43.0	40
Fixed-4	18.07	12.71	14.92	0.748	baseline		48.3	67
Fixed-8	14.54	13.41	13.95	0.979	−6.5 [−9.4, −3.4]	−30.9 [−34.2, −27.4]	73.1	120
AdaEDL	13.30	9.83	11.30	0.731	−24.3 [−25.2, −23.3]	+2.3 [1.2, 3.3]	44.7	0
CAPE-S	19.19	14.30	16.40	0.772	+9.9 [8.0, 12.0]	−3.2 [−4.8, −1.6]	52.1	89
CAPE-E	18.42	12.68	15.02	0.703	+0.7 [−1.3, 3.1]	+6.1 [4.5, 7.7]	44.4	55

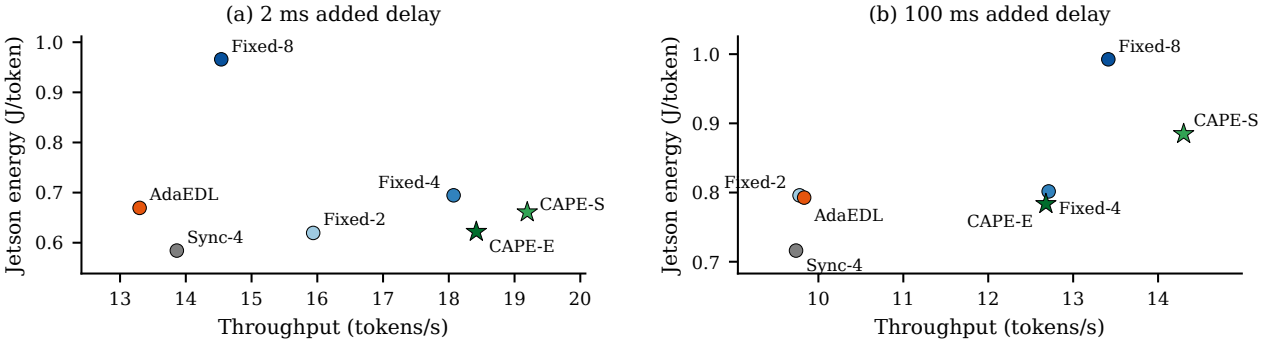


Fig. 3. Speed and energy of all methods in the main evaluation; points further right and lower are better. CAPE-S is the fastest method at both delays. CAPE-E is about as frugal as Fixed-2 at 2 ms while much faster, and has the lowest J/token of all lookahead methods at 100 ms.

TABLE II

EFFECTS OF CAPE-S AND CAPE-E RELATIVE TO FIXED-4 BY TASK (BOTH DELAYS POOLED) AND BY DELAY (ALL TASKS POOLED), WITH 95% PROMPT-CLUSTER BOOTSTRAP INTERVALS.

Method	Effect (%)	GSM8K	HumanEval	MT-Bench	2 ms	100 ms
CAPE-S	throughput gain	+8.9 [5.6, 12.1]	+17.8 [14.7, 21.3]	+5.7 [4.2, 7.6]	+6.2 [3.6, 8.9]	+12.5 [10.2, 14.9]
CAPE-S	J/token saving	−1.7 [−3.9, 0.7]	−0.4 [−2.4, 1.6]	−6.5 [−8.9, −3.8]	+4.9 [2.9, 7.0]	−10.4 [−12.3, −8.3]
CAPE-E	throughput gain	−0.3 [−2.6, 2.4]	+11.5 [7.8, 16.1]	−5.0 [−6.4, −3.2]	+1.9 [−0.4, 4.9]	−0.3 [−2.7, 2.5]
CAPE-E	J/token saving	+4.3 [2.6, 6.2]	+0.6 [−1.1, 2.5]	+11.7 [9.1, 14.6]	+10.5 [8.4, 12.5]	+2.3 [0.9, 3.6]

TABLE III

HELD-OUT SETS WITH CAPE'S DEFAULT SETTING ($w = 0.25$), 2 AND 100 MS POOLED. GAINS ARE RELATIVE TO FIXED-4 WITH 95% PROMPT-CLUSTER BOOTSTRAP INTERVALS.

Method	tok/s	J/tok	Throughput gain (%)	J/token saving (%)
Set 1: 9 prompts, 72 requests				
Fixed-4	13.44	0.849	baseline	
Regime	13.57	0.829	+0.9 [−2.4, 4.1]	+2.4 [−2.9, 6.7]
CAPE	14.09	0.817	+4.8 [1.2, 9.3]	+3.7 [−1.5, 7.6]
CAPE-E	13.44	0.791	+0.0 [−3.4, 4.5]	+6.8 [0.2, 11.2]
Set 2: 12 prompts, 72 requests				
Fixed-4	14.87	0.788	baseline	
CAPE	15.77	0.760	+6.0 [1.1, 10.7]	+3.6 [−0.6, 9.0]
CAPE-robust	15.68	0.772	+5.4 [0.4, 9.5]	+2.1 [−2.2, 7.4]

TABLE IV

OUTPUTS IN THE MAIN EVALUATION (220 REQUESTS PER METHOD): GSM8K ANSWERS CORRECT OUT OF 100, TRUNCATIONS AT 512 TOKENS, REQUEST PAIRS WHOSE TOKEN SEQUENCE DIFFERS FROM FIXED-4 (≠), AND OUTPUTS IDENTICAL TO THE CLOUD'S OWN AUTOREGRESSIVE GENERATION (= CLOUD AR).

Method	GSM8K	Truncated	≠ Fixed-4	= cloud AR
Sync-4	72/100	23/220	1	83/220
Fixed-2	72/100	24/220	20	85/220
Fixed-4	72/100	23/220	—	83/220
Fixed-8	74/100	17/220	26	84/220
AdaEDL	74/100	19/220	14	84/220
CAPE-S	73/100	23/220	19	86/220
CAPE-E	73/100	24/220	19	84/220

from it in only 1 of 220 requests. Fixed windows of other sizes differ as often as CAPE (Fixed-2: 20, Fixed-8: 26, CAPE-S: 19). This is consistent with near-tied scores of the quantized target that resolve differently for different numbers of verified positions. The same sensitivity separates speculative decoding from the cloud's own autoregressive generation: every method, including Fixed-4, reproduces it exactly in only 83–86 of 220 requests.

These differences do not affect task accuracy. GSM8K accuracy ranges from 72 to 74 correct answers out of 100 across all methods (Fixed-4: 72, CAPE-S: 73, CAPE-E: 73).

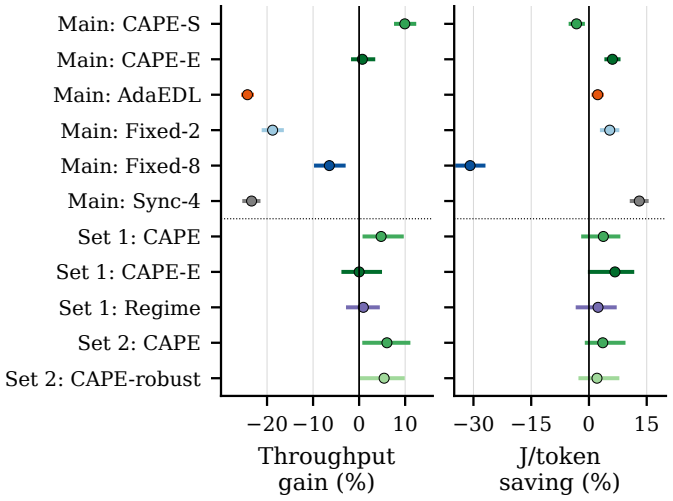


Fig. 4. Paired effects relative to Fixed-4 (2 and 100 ms pooled) with 95% prompt-cluster bootstrap intervals. Above the dotted line: main evaluation (110 prompts); below: held-out sets.

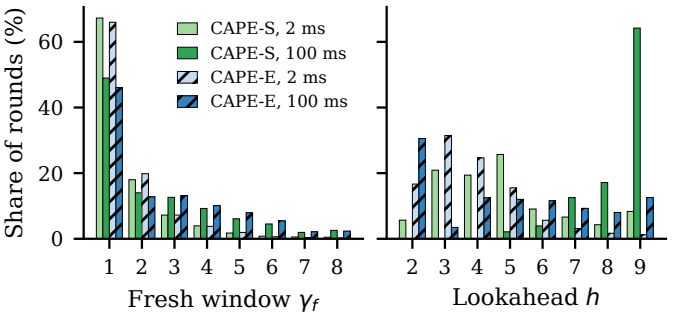


Fig. 5. Fresh windows and lookahead lengths chosen by CAPE-S and CAPE-E in the main evaluation. Fresh windows stay short; lookahead grows with delay, and less so when energy is weighted.

Truncation at the 512-token limit is equally frequent (23 of 220 requests for both Fixed-4 and CAPE-S). Choosing (γ_f, h) takes about 0.42 ms per round in Python. The controller needs no training, no draft probabilities and no change to the cloud.

VII. Discussion and Limitations

Scale. The main evaluation covers 110 prompts from three task families, with one observation per prompt, method and delay. The prompt-cluster intervals account for prompt-to-prompt variation, and the held-out sets and the subset of previously unused prompts agree with the main result. Repeated runs would additionally capture run-to-run variation of the cloud and the network.

Generality. We study one device, one model pair and one server GPU. CAPE measures its cost inputs (V, d) and acceptance rates online, so we expect it to transfer to other model pairs and devices with only the power ratio r recalibrated, but we have not yet tested this.

Network conditions. All results include the jitter of a real campus Wi-Fi path with both endpoints on wireless links, whose per-request overhead exceeded 100 ms in 12% of rounds; the median-of-three round-trip estimate absorbs such spikes. The added delays, however, are constant, and we have not evaluated cellular access networks or wide-area traces.

Energy scope and trade-off. We report gross board energy of the edge device, which includes idle power. The energy of the RTX 3080 server and of the network is not measured, so our results concern the device's energy budget, not the system's. A data-center GPU would also verify faster than a desktop RTX 3080, which shortens V and favors shorter lookahead; CAPE adapts to this because it measures V online. The speed setting CAPE-S uses 3.2% more J/token than Fixed-4 overall and 10.4% more at 100 ms. Applications that must not spend more energy per token should use $w \geq 0.25$, which gave speedups without a measurable energy penalty on the held-out sets, or $w = 0.5$, which saves energy at unchanged speed.

Calibration of r. The controller uses $r = 2.3$ from development runs. Fitting Eq. (1) to our data sets gives r between 1.8 and 3.2. Wall time and draft-busy time are correlated across requests, so the split between the two coefficients is less certain than their total, and the calibrated value lies within the fitted range. Because r only scales the energy term, a mis-calibrated r acts like a different energy weight. Online calibration from the power sensor is a natural extension.

Exactness. Strict greedy verification makes every emitted token the target's top choice, but with a quantized target near-ties resolve differently for different verification shapes. Outputs therefore differ between methods for some prompts, as they do between fixed windows of different sizes and between speculative and autoregressive decoding (Section VI-E). GSM8K accuracy was unaffected. HumanEval pass@1 and MT-Bench quality scores were not evaluated.

Cloud-side opportunity. Verification time is dominated by recomputing the context, because requests for prompt log-probabilities bypass the prefix cache in our deployment. Incremental verification with cached prefixes would shrink V substantially. This is complementary to CAPE, which would respond by shortening its lookahead, since it measures V online.

VIII. Conclusion

Edge–cloud speculative decoding pays a network round trip for every verification. Our measurements on a Jetson Orin Nano show that synchronous drafting occupies a large share of each round, that verification cost hardly depends on the draft length, and that draft work has a predictable energy price. CAPE turns these observations into a two-state Markov model of the pipeline and chooses, in every round, how many tokens to draft on the critical path and how many to prefetch in the shadow of verification. It runs entirely on the device, keeps strict greedy verification, and exposes one weight that moves it along the speed–energy frontier. On 110 prompts, its speed setting is 9.9% faster than the fastest fixed window and faster than the best fixed window for each delay, while its energy setting saves 6.1% of the energy per token at equal speed. Both settings are far faster than synchronous decoding and AdaEDL. Evaluations on more devices, model pairs and real

network traces, online calibration of the energy model and cloud-side incremental verification are the next steps.

References

[1] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," in *Proceedings of the 40th International Conference on Machine Learning*, vol. 202. PMLR, 2023, pp. 19 274–19 286. [Online]. Available: https://proceedings.mlr.press/v202/leviathan23a.html

[2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating Large Language Model Decoding with Speculative Sampling," arXiv preprint arXiv:2302.01318, 2023. [Online]. Available: https://arxiv.org/abs/2302.01318

[3] X. Li, D. Spatharakis, S. Ghafouri, J. Fan, H. Vandierendonck, D. John, B. Ji, and D. Nikolopoulos, "SLED: A speculative LLM decoding framework for efficient edge serving," in *Proceedings of the Tenth ACM/IEEE Symposium on Edge Computing (SEC)*, 2025.

[4] J. Park, S. Cho, and D. Han, "SpecEdge: Scalable Edge-Assisted Serving Framework for Interactive LLMs," in *Advances in Neural Information Processing Systems*, 2025. [Online]. Available: https://papers.nips.cc/paper_files/paper/2025/file/8587069d00a69d0ea498d547fffad6dd-Paper-Conference.pdf

[5] F. Yu, L. Li, B. McDanel, and S. Q. Zhang, "DSD: A Distributed Speculative Decoding Solution for Edge-Cloud Agile Large Model Serving," arXiv preprint arXiv:2511.21669, 2025. [Online]. Available: https://arxiv.org/abs/2511.21669

[6] Y. Zhang, Z. Gao, S. Yue, J. Li, and R. Wang, "A Pipelined Collaborative Speculative Decoding Framework for Efficient Edge-Cloud LLM Inference," arXiv preprint arXiv:2603.19133, 2026. [Online]. Available: https://arxiv.org/abs/2603.19133

[7] X. Li, J. Fan, Q. Wang, D. Spatharakis, S. Ghafouri, H. Vandierendonck, D. John, B. Ji, A. R. Butt, and D. S. Nikolopoulos, "WISP: Waste- and Interference-Suppressed Distributed Speculative LLM Serving at the Edge via Dynamic Drafting and SLO-Aware Batching," arXiv preprint arXiv:2601.11652, version 2, 2026. [Online]. Available: https://arxiv.org/abs/2601.11652v2

[8] T. Liu, Y. Li, Q. Lv, K. Liu, J. Zhu, W. Hu, and X. Sun, "PEARL: Parallel Speculative Decoding with Adaptive Draft Length," in *International Conference on Learning Representations*, 2025. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/hash/03b1043052700b1a471996b0baf309d4-Abstract-Conference.html

[9] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. [Online]. Available: https://arxiv.org/abs/2309.06180

[10] S. Agrawal, W. Jeon, and M. Lee, "AdaEDL: Early draft stopping for speculative decoding of large language models via an entropy-based lower bound on token acceptance probability," in *Proceedings of the 4th NeurIPS Efficient Natural Language and Speech Processing Workshop*, ser. Proceedings of Machine Learning Research, vol. 262, 2024. [Online]. Available: https://proceedings.mlr.press/v262/agrawal24a.html

[11] H. Xia, Z. Yang, Q. Dong, P. Wang, Y. Li, T. Ge, T. Liu, W. Li, and Z. Sui, "Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 7655–7671.

[12] X. Miao *et al.*, "SpecInfer: Accelerating generative large language model serving with tree-based speculative inference and verification," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.

[13] Z. Chen, A. May, R. Svirschevski, Y. Huang, M. Ryabinin, Z. Jia, and B. Chen, "Sequoia: Scalable, robust, and hardware-aware speculative decoding," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[14] S. Kim, K. Mangalam, S. Moon, J. Malik, M. W. Mahoney, A. Gholami, and K. Keutzer, "Speculative decoding with big little decoder," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[15] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, "Medusa: Simple LLM inference acceleration framework with multiple decoding heads," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024, pp. 5209–5235.

[16] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE: Speculative sampling requires rethinking feature uncertainty," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024, pp. 28 935–28 948.

[17] Y. Fu, P. Bailis, I. Stoica, and H. Zhang, "Break the sequential dependency of LLM inference using lookahead decoding," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024, pp. 14 060–14 079.

[18] K. Huang, X. Guo, and M. Wang, "SpecDec++: Boosting speculative decoding via adaptive candidate lengths," in *Conference on Language Modeling (COLM)*, 2025. [Online]. Available: https://arxiv.org/abs/2405.19715

[19] D. Xu, W. Yin, H. Zhang, X. Jin, Y. Zhang, S. Wei, M. Xu, and X. Liu, "EdgeLLM: Fast on-device LLM inference with speculative decoding," *IEEE Transactions on Mobile Computing*, vol. 24, no. 4, pp. 3256–3273, 2025.

[20] Y. Lyu, B. Irukulapati, and J. P. Champati, "Speculation at a distance: Where edge-cloud speculative decoding actually pays off," arXiv preprint arXiv:2606.25091, 2026. [Online]. Available: https://arxiv.org/abs/2606.25091

[21] S. Samsi, D. Zhao, J. McDonald, B. Li, A. Michaleas, M. Jones, W. Bergeron, J. Kepner, D. Tiwari, and V. Gadepally, "From words to watts: Benchmarking the energy costs of large language model inference," in *IEEE High Performance Extreme Computing Conference (HPEC)*, 2023, pp. 1–9.

[22] R. Dutta, P. Koley, S. Poddar, J. Misra, S. Podder, N. Balani, S. Ghosh, and N. Ganguly, "Benchmarking the Energy Savings with Speculative Decoding Strategies," in *Findings of the Association for Computational Linguistics: EACL 2026*. Association for Computational Linguistics, 2026, pp. 4737–4748. [Online]. Available: https://aclanthology.org/2026.findings-eacl.249/

[23] A. Grattafiori *et al.*, "The Llama 3 herd of models," arXiv preprint arXiv:2407.21783, 2024. [Online]. Available: https://arxiv.org/abs/2407.21783

[24] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," in *Proceedings of Machine Learning and Systems*, vol. 6, 2024. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2024/hash/42a452cbafa9dd64e9ba4aa95cc1ef21-Abstract-Conference.html

[25] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training Verifiers to Solve Math Word Problems," arXiv preprint arXiv:2110.14168, 2021. [Online]. Available: https://arxiv.org/abs/2110.14168

[26] M. Chen, J. Tworek, H. Jun, Q. Yuan *et al.*, "Evaluating large language models trained on code," arXiv preprint arXiv:2107.03374, 2021. [Online]. Available: https://arxiv.org/abs/2107.03374

[27] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, vol. 36, 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html

Appendix A
Reproducibility

All numbers, tables and figures in this paper are generated from retained raw records by `scripts/extract_evidence.py` and `scripts/make_assets.py`; the text reads every value from the generated `numbers.tex`. Only the 2 ms and 100 ms conditions are used. The raw records, power traces, frozen plans and source snapshots are in the repository:

- Main evaluation: `results/compare_20261010/full_110` (plan `full_110_plan.json`; host activity during the run is logged in `full_110_maintenance.log`);
- Held-out Set 1: `results/pipeline_research_20261009/confirm_markov`;
- Held-out Set 2: `results/pipeline_research_20261009/final_markov_v2`.

Models and software: draft `bartowski/Llama-3.2-1B-Instruct-GGUF` (Q4_K_M) on llama.cpp build b9744 with JetPack 6.2.1; target `hugging-quants/Meta-Llama-3.1-8B-Instruct-AWQ-INT4` on vLLM 0.26.0 and PyTorch 2.11.0 (conda environment) with an RTX 3080 (10 GB), NVIDIA driver 580.173.02, Ubuntu 24.04.4 LTS (kernel 7.0.0-28). Method identifiers in the raw records map to the names used here as follows.

Record	Paper	Record	Paper
`sync_g4`	Sync-4	`markov`	CAPE ($w = 0.25$)
`fixed_g2`	Fixed-2	`markov_w0`	CAPE-S ($w = 0$)
`fixed_fast`	Fixed-4	`markov_w05`	CAPE-E ($w = 0.5$)
`fixed_g8`	Fixed-8	`markov_v2`	robust variant
`adaedl`	AdaEDL	`regime_short`	Regime

The controller is in `edge/edge_client_pipeline.py` and `common/pipeline_model.py`, the AdaEDL baseline in `edge/edge_client_adaedl.py`, and the experiment runner in `experiments/progress_probe.py`.